ASP.NET Core web server converts the representation to a raw HTTP response and sends it back to the reverse proxy, which forwards it to the user's browser.

As you saw in chapter 3, you define the middleware pipeline in code as part of your initial application configuration in Program.cs. You can tailor the middleware pipeline specifically to your needs; simple apps may need only a short pipeline, whereas large apps with a variety of features may use much more middleware. Middleware is the fundamental source of behavior in your application. Ultimately, the middleware pipeline is responsible for responding to any HTTP requests it receives.

Requests are passed to the middleware pipeline as `HttpContext` objects. As you saw in chapter 3, the ASP.NET Core web server builds an `HttpContext` object from an incoming request, which passes up and down the middleware pipeline. When you're using existing middleware to build a pipeline, this detail is one that you'll rarely have to deal with. But as you'll see in the final section of this chapter, its presence behind the scenes provides a route to exerting extra control over your middleware pipeline.

You can also think of your middleware pipeline as being a series of concentric components, similar to a traditional matryoshka (Russian) doll, as shown in figure 4.3. A request progresses through the pipeline by heading deeper into the stack of middleware until a response is returned. Then the response returns through the middleware, passing through the components in reverse order from the request.